Multi-Agent Decision OS Development Report

生成日期：2026-04-10

作者：Codex

项目位置：`C:\Users\CLH`

报告目的：为后续新对话继续开发提供完整背景、现状、设计逻辑、已确认决策与后续实施方向。

---

1. 报告摘要

本项目是一套面向 Mark 的“多 Agent / 多 skill 决策操作系统”。当前已经具备较完整的 `/debate` 结构化圆桌会议模式，能够在用户显式触发时，基于本地名人 skill 组织一次有分工、有主持、有审查、有统一决议的多 Agent 讨论。

本轮讨论在现有 `/debate` 基础上，新增了第二种交互形态 `/room` 的产品与系统设计方向。`/room` 不替代 `/debate`，而是补足“持续协同推演、半结构化群聊、按阶段推进复杂问题”的使用场景。该模式的核心不是闲聊，而是协同推演。

本报告将完整记录：

- 现有 `/debate` 系统的定位与约束
- 当前本地名人 skill 体系与 Agent 映射方式
- 为什么需要新增 `/room`
- `/room` 的产品定位、用户交互和后台筛选逻辑
- 新增名人 skill 时的自动发现与条件注册机制
- 半结构化评分系统的详细设计
- 当前已经确认的设计决议
- 仍未完成的开发项与建议实现顺序

---

2. 开发背景

2.1 已有项目背景

用户已在本地构建了一套多名人 skill 系统，包含但不限于：

- `steve-jobs-skill`
- `elon-musk-skill`
- `munger-skill`
- `feynman-skill`
- `naval-skill`
- `taleb-skill`
- `zhangxuefeng-skill`
- `paul-graham-skill`
- `zhang-yiming-skill`
- `karpathy-skill`
- `ilya-sutskever-skill`
- `mrbeast-skill`
- `trump-skill`

这些 skill 在原始设计中不是“聊天人格”，而是“认知插件”或“认知镜头”。核心原则是先判断问题类型，再选择对应的认知镜头，而不是把名人当成娱乐性人设。

2.2 已有 `/debate` 背景

用户此前已经要求 Codex 开发过一套圆桌调度系统，其显式触发词为 `/debate`。本轮已确认这套机制不是新设想，而是已有项目中的正式结构。

`/debate` 的定位是：

- 仅在用户显式输入 `/debate` 时触发
- 作为上层调度 skill 工作
- 组织 3-5 个最合适的名人 Agent 参会
- 先分工，再发言，再汇总，再审查，再给统一决议
- 不污染日常单 skill 路由
- 不让多 Agent 协作变成松散群聊

2.3 新增 `/room` 的动机

用户在本轮对话中进一步明确了一个新需求：

> 讨论复杂问题时，希望像一个“Agent team / 多 Agent 聊天室”那样，把本地名人 skill 分配给不同 Agent，让它们持续协同，像一个顾问群一样参与复杂问题的推演。

用户并不想让系统只有正式的 `/debate` 会议模式，还希望有一种更灵活、更像多人群聊、但仍然可控的形态。因此，新增 `/room` 成为本轮设计的核心任务。

---

3. 现有 `/debate` 系统复盘

3.1 系统使命

根据 `C:\Users\CLH\AGENTS.md`，项目使命是构建一套面向 Mark 的多 Agent / 多 skill 决策操作系统，分为两层：

- 日常层：按问题类型调用单个或少量名人 skill
- 圆桌层：显式输入 `/debate` 时进入结构化多 Agent 圆桌会议

3.2 `/debate` 的核心原则

当前系统强调：

- 不按“想和谁聊天”路由，而按问题类型路由
- 所有 skill 都是认知镜头，不是真人本体
- 多 Agent 协作必须严格区分事实、推断、不确定项、建议
- 必须有主持与审查，不能无边界扩写
- 最终必须给出统一决议，不能把多个答案并列甩给用户

3.3 `/debate` 的任务类型

`/debate` 当前正式使用 8 类路由：

- `startup`
- `product`
- `learning`
- `content`

- `risk`
- `planning`
- `strategy`
- `writing`

3.4 `/debate` 的默认组合

当前已经有写死的默认组合，例如：

- `startup`：Paul Graham + Steve Jobs + Munger + Taleb
- `product`：Steve Jobs + Zhang Yiming + Elon Musk + Munger
- `learning`：Karpathy + Feynman + Ilya
- `strategy`：Paul Graham + Munger + Taleb + Zhang Yiming

3.5 `/debate` 的工作流程

当前正式流程包括：

1. 识别议题
2. 识别任务类型
3. 选择参会 Agent
4. 分配职责
5. 逐个发言
6. 主持人汇总
7. 审查 Agent 审核
8. 输出最终决议

3.6 `/debate` 的边界

`/debate` 明确不是自由聊天型群聊，而是高度结构化的“重大议题会议模式”。这一点在新增 `/room` 设计时必须保留，不可破坏。

---

4. 为什么 `/debate` 不等于“多 Agent 聊天室”

本轮讨论已经明确区分：

4.1 `/debate` 的本质

`/debate` 更像：

- 董事会会议
- 正式圆桌
- 高严肃度决策收敛流程

特征是：

- 先分工
- 按顺序发言
- 默认一轮或有限轮
- 主持器强组织
- 审查后才能放行
- 最后只给单一结论

4.2 多 Agent 聊天室的本质

用户想要的“Agent team / 多 Agent 聊天室”更像：

- 专家群聊
- 顾问群
- 可以持续存在的半结构化讨论房间

特征是：

- 更自然的群聊感
- 用户可随时插话
- Agent 可按需互相补位
- 可持续保留上下文
- 不一定每轮都强制走正式审查

由此得出关键结论：

> `/debate` 属于多 Agent 协作系统，但不是“开放式多 Agent 聊天室”。

因此需要单独新增 `/room`。

---

5. `/room` 的正式产品定位

5.1 用户需求权重

用户明确给出的用途权重为：

- 协同推演：60%
- 探索想法：20%
- 日常陪跑：20%

因此 `/room` 的核心定位不是“闲聊”，而是：

> 一个以协同推演为核心目标的半结构化多 Agent 房间。

5.2 `/room` 的一句话定义

`/room` 是一个持续可插话、可点名、可阶段总结、可按需升级为 `/debate` 的半结构化多 Agent 房间。

5.3 `/room` 与 `/debate` 的关系

两者必须长期共存，且边界清晰：

- `/room`：负责探索、推演、澄清、反复推进
- `/debate`：负责正式裁决、风险审查、统一决议

5.4 产品角色分工

可以把两者理解为：

- `/room` = 顾问群
- `/debate` = 董事会会议

`/room` 不是削弱版 `/debate`，而是另一种协作形态。

---

6. `/room` 采用的总体方案

本轮已确认采用的设计方向是：

> 方案 3：混合双层房间

6.1 该方案的含义

`/room` 默认是轻量聊天室：

- 你说一句
- 系统挑 2-4 个最相关 Agent 回应
- 允许 1-2 轮简短互相补位

当议题变复杂、冲突变大、需要更强收敛时，可以：

- 手动收窄子议题
- 要求阶段总结
- 切到更聚焦的推演模式
- 进一步升级为 `/debate`

6.2 该方案优点

- 保留群聊感
- 不会退化成完全自由乱聊
- 不会和现有 `/debate` 冲突
- 适合复杂问题在多轮中逐步清晰化

---

7. `/room` 的命令语义设计

本轮已设计的主要命令语义如下。

7.1 建房命令

- `/room 议题`
- `/room --with Jobs,Taleb 议题`
- `/room --without Trump 议题`
- `/room --topic product 议题`
- `/room --resume 房间名`
- `/room --persistent 议题`

7.2 房间内命令

- `@Jobs ...`
- `/focus 子问题`
- `/expand`

- `/summary`
- `/status`
- `/agents`
- `/add Taleb`
- `/remove MrBeast`
- `/decide`
- `/upgrade-to-debate`

7.3 关键交互原则

- 默认临时房间，避免上下文污染
- 只有显式 `--persistent` 或 `/resume` 才保留长期房间状态
- 任何时候都可以点名单个 Agent 回应
- 支持按子问题切换讨论焦点

---

8. `/room` 的房间状态模型

本轮已确认 `/room` 至少应维护以下状态：

- `room_id`
- `title`
- `mode`
- `primary_type`
- `secondary_type`
- `agents`
- `agent_roles`
- `active_focus`
- `conversation_log`
- `consensus_points`
- `open_questions`
- `tension_points`
- `recommended_next_step`
- `upgrade_signal`

8.1 设计原则

`/room` 的状态不是为了存档而存档，而是为了推进讨论。

系统必须始终知道：

- 当前在讨论什么
- 已经明确了什么
- 还没明确什么
- 接下来建议推进哪一步

---

9. `/room` 的 Agent 来源原则

本轮已经明确：

> `/room` 不创建新 Agent 人格，必须复用用户本地已经配置好的名人 skill。

也就是说：

- `Agent = 本地名人 skill 的房间化实例`
- 展示层使用 `Jobs`、`Taleb`、`Karpathy` 等简称
- 底层映射到精确 skill 名

例如：

- `Jobs` -> `steve-jobs-skill`
- `Taleb` -> `taleb-skill`
- `Karpathy` -> `karpathy-skill`

9.1 与 `/debate` 的一致性要求

`/room` 和 `/debate` 应共享：

- 同一套 skill 映射表
- 同一套角色边界
- 同一套标签体系
- 同一套对冲逻辑

这保证它们是同一个系统中的两种模式，而不是两套平行系统。

---

10. 新增 skill 时的自动发现与条件注册机制

这是本轮讨论中的关键新增需求。

10.1 当前系统的问题

当前 `/debate` 的可用名人 skill 列表是写死在文档里的，因此：

- 新增 skill 后不会自动进入 `/debate` 或未来 `/room` 的候选池
- 必须手工更新文档和路由

用户要求改成：

> 自动发现 + 条件注册

10.2 设计目标

新 skill 加入本地目录后：

- 系统应自动发现它的存在
- 但不是发现就直接让它参会
- 只有满足协作要求时才进入 Agent Registry

10.3 三种注册状态

建议引入：

- `registered`
- `discovered_but_incomplete`
- `disabled`

10.4 注册要求

一个 skill 要想被纳入 `/room` 和 `/debate` 的自动选人池，至少要具备：

- `display_name`
- `skill_name`
- `short_name`
- `primary_strengths`
- `non_goals`
- `task_types`

- `stance_tags`
- `expression_tags`
- `dominance_level`
- `counterbalance_with`
- `room_profile`
- `debate_profile`

10.5 推荐实现方式

优先读取：

- `roundtable-profile.md`
- 或未来标准化的 `agent-profile.json`

如果缺少这些信息：

- skill 进入 `discovered_but_incomplete`
- 可以被系统发现
- 但不能自动参会

10.6 核心原则

> 不是装上 skill 就自动上麦，而是先发现，再检查，再注册。

这样可以防止系统被未经建模的新 skill 污染。

---

11. 为什么不是所有名人都参会

用户明确要求：

> 后台应该有一套筛选机制，根据当前讨论主题，选择最适合的人选参会，而不是让所有人都进来。

这一点非常关键。系统不是为了“热闹”，而是为了“有效推进”。

由此本轮得出的根本原则是：

> 不是问“谁最有名”，而是问“当前这一步最需要谁”。

---

12. 参会筛选机制的总体思路

本轮已设计的后台选人流程如下：

1. 议题解析
2. 候选召回
3. 相关性打分
4. 模型语义校正
5. 结构平衡校验
6. 人数裁剪与职责分配

12.1 议题解析时抽取的信号

系统需要先抽取：

- 主任务类型
- 当前子问题
- 当前讨论阶段
- 用户显式偏好

这一步确保“同样是 product，也能分辨是在问切口、增长、风险还是体验”。

12.2 候选召回

先根据类型、子问题和阶段召回可能相关的 Agent，而不是直接最终定人。

12.3 最终目标

最终选人不是为了组成“最豪华阵容”，而是为了组成：

> 当前这一轮最小够用的最佳阵容。

---

13. 半结构化评分系统设计

这是本轮最关键的技术设计之一。

13.1 总原则

用户追问的核心问题是：

> 相关性打分的核心标准是什么？是不是交给模型自己判断？

本轮结论是：

> 不应让模型自由拍脑袋选人，而应采用“规则打底，模型补充判断”的半结构化评分系统。

13.2 总体流水线

建议后台按如下流程工作：

1. 解析当前输入
2. 硬约束过滤
3. 结构化打分
4. 模型小范围语义校正
5. 结构平衡
6. 人数裁剪

---

14. 第一层：硬约束过滤

这一层不算分，只做“能不能进池”。

过滤掉：

- 不在 `registered` 状态的 skill
- 被 `--without` 显式排除的 Agent
- 明显不适合当前大类问题的 Agent
- 在 `non_goals` 中与当前子问题正面冲突的 Agent
- 高重复、且当前没有新增价值的 Agent

14.1 作用

这一步的目的是减少无效候选，避免后续打分阶段被大量噪音干扰。

---

15. 第二层：结构化打分

本轮已确认总分建议拆成 7 个分项，满分 100。

15.1 子问题匹配分 `0-30`

这是最高权重。

系统问的不是：

- 他适不适合这个大类？

而是：

- 他适不适合回答当前这个具体子问题？

例如：

- “值不值得做” -> Paul Graham / Jobs / Munger
- “尾部风险” -> Taleb / Munger
- “实现路径” -> Karpathy / Ilya

15.2 任务类型匹配分 `0-20`

看他是否擅长当前主类型 / 副类型。

15.3 阶段适配分 `0-15`

当前阶段可以粗分为：

- `explore`
- `simulate`
- `stress_test`
- `converge`
- `decision`

不同阶段适合不同 Agent。

15.4 职责独特性分 `0-15`

衡量这个 Agent 带来的信息增量。

如果房间里已经有人承担相近职责，则独特性下降。

15.5 结构补位分 `0-10`

看他能不能补足当前房间短板。

例如：

- 当前缺防守位
- 当前缺 grounded 位
- 当前缺现实校准位

15.6 用户偏好分 `-10 到 +10`

响应：

- `--with`
- `--without`
- `@Taleb`
- 用户显式排斥或偏爱某种视角

15.7 冗余与越权惩罚 `-20 到 0`

惩罚：

- 与现有主责高度重复
- 当前问题不在其擅长范围
- 会严重破坏房间结构平衡

---

16. 评分公式

本轮建议的公式为：

```
TotalScore =
  子问题匹配分
+ 任务类型匹配分
+ 阶段适配分
+ 职责独特性分
+ 结构补位分
```

```
+ 用户偏好分
- 冗余与越权惩罚
```

建议默认权重：

- 子问题匹配：30
- 任务类型匹配：20
- 阶段适配：15
- 职责独特性：15
- 结构补位：10
- 用户偏好：10
- 冗余惩罚：-20

该权重体现的原则是：

- 最重要的是“当前问题需不需要你”
- 其次是“这个问题类型你是不是确实擅长”
- 用户偏好重要，但不能压过问题本身

---

17. 模型在评分中的角色

本轮明确：

> 模型不直接决定参会名单。

模型只负责三类事情：

17.1 语义解析

把自然语言转成结构化输入：

- 主类型
- 副类型
- 子问题
- 阶段
- 显式约束

17.2 软标签判断

例如：

- 当前更像真假判断还是执行推演
- 当前更需要解释还是风险压测

17.3 小范围校正

允许模型做 `-5 到 +5` 的小范围调整，但不能：

- 绕过硬约束
- 把低基础分角色硬拉到第一
- 推翻结构平衡规则

并且每次校正都应该附带简短理由。

---

18. 最终不是“分高就上”，而是组合优化

这是本轮讨论中特别强调的一点。

系统不是简单取前 3 或前 5 高分，而要考虑：

- 阵容是否平衡
- 是否有职责冲突
- 是否有明显重复
- 是否缺少防守位
- 是否缺少 grounded 位

建议最终组合满足：

- 至少 1 个主推进位
- 至少 1 个防守位
- 至少 1 个 grounded 位
- dominant 不超过一半
- 主责不可重复

因此，最后的参会名单应由“个人高分”与“组合健康度”共同决定。

---

19. 已确认的关键设计决议

截至本次对话，已经明确确认的设计决议如下。

19.1 关于 `/room`

- `/room` 新增为正式产品形态
- 定位是“半结构化多 Agent 房间”
- 核心用途权重为：协同推演 60%，探索 20%，日常陪跑 20%
- `/room` 不替代 `/debate`

19.2 关于 `/debate`

- `/debate` 保持原有正式圆桌与审查机制
- 继续作为重大议题的高严肃度模式存在

19.3 关于 Agent 来源

- `/room` 和 `/debate` 都必须复用本地名人 skill
- 不新造一套独立人格体系

19.4 关于新增 skill

- 支持自动发现
- 采用条件注册，不是安装即参会

19.5 关于选人

- 不让所有名人都参会
- 按议题、子问题、阶段和房间结构进行筛选

19.6 关于评分系统

- 使用半结构化评分系统
- 规则打底，模型辅助
- 不允许模型自由拍脑袋决定最终名单

---

20. 当前仍未完成的设计部分

以下内容在本轮尚未设计完成，属于下一轮开发应优先继续的部分。

20.1 `/room` 的发言机制

需要继续定义：

- 每轮默认让几个 Agent 发言
- 是否允许 Agent 互相引用
- 允许互相回应的轮数上限
- 如何避免房间变成多人格表演

20.2 换人机制

需要继续定义：

- 哪些 Agent 是“常驻核心组”
- 什么时候临时补人
- 什么时候临时降权或移出

20.3 升级机制

需要继续定义：

- 什么时候从 `/room` 建议升级到 `/debate`
- 升级时如何把当前房间状态压缩成 `/debate` 审查包输入

20.4 状态持久化

需要继续定义：

- persistent room 的状态存在哪
- 是否按房间名、日期、主题落盘
- 如何恢复状态

20.5 Agent Registry 文件格式

需要继续定义：

- 用 `md`、`json` 还是混合格式
- 自动扫描目录规则
- 注册缓存机制

20.6 选人打分的实现细节

需要继续定义：

- 结构化分数字段如何序列化
- 模型小范围校正如何记录理由
- 是否保留历史有效性分

---

21. 建议的后续开发顺序

为了保证项目可持续推进，建议下一轮开发按下面顺序进行。

Phase 1. 定义 `/room` 协议

先补完：

- 发言机制
- 换人机制
- 升级到 `/debate` 的规则

Phase 2. 定义 Agent Registry

补完：

- 自动发现规则
- 条件注册标准
- 元数据文件格式

Phase 3. 定义 Selection Policy

把本报告中已有的评分系统写成正式规则文档。

Phase 4. 补文档入口

建议新增：

- `docs/room-architecture.md`
- `docs/agent-registry.md`
- `docs/room-selection-policy.md`
- `prompts/room-chat.md`
- `prompts/room-summary.md`
- `prompts/room-upgrade.md`

Phase 5. 再决定实现载体

是继续作为文档与 prompt 驱动的 skill 系统，还是补一个真正的状态管理实现层。

---

22. 建议新增的正式文档

下一轮建议把当前报告拆分沉淀成这些长期文件：

- `docs/room-product-spec.md`
- `docs/room-state-model.md`
- `docs/agent-registry.md`
- `docs/selection-policy.md`
- `docs/room-to-debate-handoff.md`
- `prompts/room-chat.md`
- `prompts/room-summary.md`
- `prompts/room-upgrade.md`

这样后续新增 skill、改路由、改评分系统时，都能在清晰边界内维护。

---

23. 开发注意事项

23.1 必须保持 `/debate` 边界稳定

不要为了做 `/room` 把 `/debate` 变成聊天室。

23.2 不要让新增 skill 自动污染系统

必须坚持自动发现 + 条件注册，而不是发现即参会。

23.3 选人系统必须能解释

未来实现时，系统应该尽量能解释：

- 为什么选这几个人
- 为什么没选另外几个人
- 当前每个人的职责是什么

否则后期会很难维护。

23.4 强调“最小够用阵容”

不要让系统默认“人越多越好”。多 Agent 的核心价值是结构化互补，不是人数。

---

24. 可直接给下一轮对话的简短交接摘要

如果后续要在新对话里继续开发，可以直接把下面这段作为项目交接摘要：

本项目已存在正式的 `/debate` 多 Agent 圆桌会议系统，基于本地名人 skill 工作，强调结构化分工、主持汇总、审查放行和统一决议。当前新增需求是设计第二种模式 `/room`，用于半结构化多 Agent 房间，核心用途是协同推演，权重为协同推演 60%、探索 20%、日常陪跑 20%。`/room` 必须复用本地名人 skill，不创建新人格；新增 skill 采用自动发现 + 条件注册机制；选人不是全员参会，而是根据议题、子问题、阶段和房间结构做后台筛选。后台筛选已设计为半结构化评分系统：硬约束过滤 + 结构化打分 + 模型小范围校正 + 结构平衡 + 人数裁剪。当前已完成产品方向、命令语义、房间状态、Agent Registry 与评分逻辑设计，尚未完成 `/room` 发言机制、换人机制、升级到 `/debate` 的交接规则与落地文档拆分。

---

25. 文件依据

本报告综合了以下已有文件与本轮对话中的新增设计：

- `C:\Users\CLH\AGENTS.md`
- `C:\Users\CLH\docs\debate-skill-architecture.md`
- `C:\Users\CLH\docs\agent-role-map.md`
- `C:\Users\CLH\docs\reviewer-protocol.md`
- `C:\Users\CLH\prompts\debate-roundtable.md`
- 本轮关于 `/room` 的新增设计讨论

---

26. 结论

本轮开发工作本质上不是实现代码，而是完成了一个大型系统的第二形态设计定向：

- 现有 `/debate` 已被重新确认和稳固
- 新增 `/room` 的必要性已明确
- `/room` 的核心定位、与 `/debate` 的边界、Agent 来源、注册逻辑和选人评分框架都已形成清晰设计

这意味着下一轮开发可以不再重复讨论“要不要做 `/room`”，而是直接进入：

- 协议补全
- 文档拆分
- 交互细化
- 状态管理设计
- 自动发现与注册落地

本报告应视为当前阶段的正式设计备份与跨对话开发交接文档。

---

27. `/room` 的发言机制设计

这是下一轮开发里最关键的交互层设计。本轮补充的目标是让 `/room` “像群聊”，但不退化成一堆名人互相表演或重复发言。

27.1 总体原则

`/room` 的发言机制必须同时满足四个目标：

- 有群聊感
- 有信息增量
- 有节奏控制
- 能持续推进问题，而不是制造噪音

因此，本项目不应采用“所有 Agent 自由抢话”的机制，而应采用：

> 主持器控制下的轻量多轮群聊机制。

也就是说，表面上看起来像群聊，但底层仍然是被调度的。

27.2 每轮默认发言人数

建议默认规则：

- 用户每次新输入后，默认只让 `2-4` 个最相关 Agent 发言
- 轻量状态优先 `2-3` 个
- 只有在冲突明显、问题复杂时才扩到 `4` 个
- 单轮不建议超过 `4` 个，否则阅读负担明显上升

原因：

- 太少无法形成协同推演
- 太多会产生重复、争抢和阅读疲劳

27.3 发言角色分层

每轮发言建议分成三类角色：

- `primary responder`

  本轮最主要回答当前子问题的人

- `support responder`

  补充一个不同但相关的视角

- `challenge responder`

  负责指出潜在漏洞、过度自信或被忽略的问题

- `optional synthesizer`

  仅在有必要时，由主持器或某个指定 Agent 做一句阶段收束

默认情况下，一轮里不一定四类都齐全，但至少应有：

- 一个主回应位
- 一个补充或挑战位

27.4 单条发言长度约束

为了保留群聊感，`/room` 的发言必须显著短于 `/debate`。

建议设置两档：

- `chat-compact`

  1 个核心结论 + 1-2 个依据 + 1 个建议动作

- `chat-extended`

  在冲突较大或用户点名追问时，允许多写一段，但仍禁止写成正式长文

建议默认目标：

- 单个 Agent 每轮尽量控制在 `80-180` 中文字左右
- 只有用户明确要求深挖，或当前子问题确实复杂，才允许超过这一范围

27.5 Agent 是否允许互相引用

允许，但必须受限。

建议规则：

- 默认允许 Agent 对上一轮已输出的某个观点做“有限回应”
- 只允许回应“可见输出”，不允许假装看到了隐藏推理
- 回应时必须明确引用对象，例如：
  - “我同意 Jobs 关于价值主张过宽的判断，但我补一个风险点”
  - “我不同意 PG 现在就判断需求成立，因为我们还没看切换成本”

27.6 互相回应的轮数上限

为避免房间失控，建议采用“最多 2 跳”的规则。

即：

- 用户发言后，系统选出第一轮回应者
- 若有必要，允许其中 `1-2` 个 Agent 再做一轮回应或补位
- 之后必须交给：
  - 主持器做阶段总结
  - 或用户继续推进
  - 或执行 `/focus`

不允许 Agent 无限来回争论。

27.7 用户点名优先权

用户点名必须具有高优先级。

例如：

- `@Taleb 你只看风险`
- `@Jobs 回应一下 PG`

点名规则建议如下：

- 被点名的 Agent 自动进入下一轮发言名单
- 若点名内容明确限定职责，则该轮以该职责为准
- 若点名会破坏结构平衡，也不应禁止，但主持器应显式提示“当前这轮偏斜”

27.8 主持器的隐性职责

虽然 `/room` 看起来像群聊，但主持器仍需承担以下后台职责：

- 控制谁该说、谁不该说
- 压制重复发言
- 在冲突开始循环时及时收束
- 记录当前结论、分歧和待定问题
- 必要时提示用户是否应该 `/focus` 或 `/upgrade-to-debate`

27.9 防止“多人格表演”的规则

这是 `/room` 最重要的防污染机制之一。

必须禁止：

- 为了模仿名人语气而牺牲判断质量
- 为了“像群聊”而制造无意义互怼
- 多个 Agent 用不同措辞说同一个意思
- Agent 过度表演人格，不推进议题

建议明确要求：

> `/room` 中的名人 Agent 是认知角色，不是表演角色。风格可以存在，但信息增量优先级永远高于风格完整度。

27.10 阶段总结触发条件

以下情况建议自动触发主持器做简短总结：

- 连续 3 轮没有新增信息
- 出现重复冲突
- 当前子问题已经接近收敛
- 用户显式请求 `/summary`
- 系统判断当前应该切换到下一个子问题

阶段总结必须输出：

- 当前结论
- 主要分歧
- 仍未解决的问题
- 建议下一步

---

28. `/room` 的换人机制设计

如果 `/room` 要真正适合复杂问题推演，就不能采用“一组选手打完全场”的模式。换人机制是必要能力。

28.1 换人机制的核心原则

系统要区分：

- `核心常驻组`
- `阶段性参与者`
- `临时补位者`

也就是说，一个房间可以有连续上下文，但不代表每个子问题都要由同一批 Agent 负责。

28.2 核心常驻组

建议每个房间维持一个 `2-3` 人的核心常驻组。

核心常驻组的特点：

- 与议题主类型强相关
- 在整场讨论里持续有用
- 承担房间认知骨架作用

例如一个 `product + strategy` 房间，常驻组可能是：

- Jobs
- Munger
- Paul Graham

28.3 阶段性参与者

某些 Agent 只在特定阶段价值高。

例如：

- MrBeast 在“传播与选题张力”阶段进入
- Taleb 在“风险压测”阶段进入
- Karpathy 在“技术可行性”阶段进入

阶段性参与者不需要长期驻场。

28.4 临时补位者

当房间出现结构短板时，可以临时加人。

例如：

- 讨论过于进攻型，需要补一个风险位 -> 加 Taleb
- 讨论过于抽象，需要补一个接地气位 -> 加 Zhang Xuefeng
- 讨论技术路线时缺乏实现直觉 -> 加 Karpathy

28.5 什么时候应触发换人

建议触发条件包括：

- 当前子问题与现有阵容不再匹配
- 讨论阶段切换
- 当前房间出现明显结构偏斜
- 用户显式 `/add` 或 `/remove`
- 某个 Agent 连续多轮没有信息增量

28.6 什么时候该移出或降权某个 Agent

以下情况建议降权或暂时移出：

- 连续 2-3 轮没有新增信息
- 其主责已完成
- 当前子问题明显超出其边界
- 房间信息重复度过高
- 他继续发言会让房间结构失衡

注意：

- “移出”不等于删除历史观点

- 只是表示当前轮次不再优先让其继续发言

28.7 自动换人与用户可控的关系

建议采用“自动推荐 + 用户覆盖”的模式。

即：

- 系统可提示“建议加入 Taleb 压测风险”
- 系统可提示“MrBeast 在当前子问题上价值已下降，建议暂时退场”
- 但用户仍然可以手动保留、加人或点名

28.8 换人时必须解释理由

每次系统建议换人，最好能解释：

- 为什么建议加这个人
- 为什么建议暂时减少另一个人发言
- 这会补什么结构短板

否则长期使用时，用户会感觉系统在“神秘抽人”。

---

29. 从 `/room` 升级到 `/debate` 的交接协议

这是两种模式共存的核心接口，必须设计清楚。

29.1 升级的本质

`/room -> /debate` 不是重新开始，而是：

> 把房间里已经推演出的上下文，压缩成一份适合正式圆桌审议的输入包。

也就是说，升级不是丢弃 `/room`，而是把 `/room` 的成果转成 `/debate` 的正式前置材料。

29.2 什么时候建议升级

以下情况建议主持器提示升级：

- 议题已经进入“该不该定案”的阶段
- 当前分歧较大，需要正式收敛

- 当前问题代价较高，不宜继续松散讨论
- 需要显式风险审查
- 需要最终只有一个统一建议
- 用户明确输入 `/upgrade-to-debate`

29.3 什么时候不建议升级

以下情况通常不应急着升级：

- 问题仍然模糊，边界不清
- 当前还处在发散探索阶段
- 仍缺少关键背景信息
- 讨论尚未形成足够清晰的子问题

29.4 升级前必须生成的 Handoff Packet

建议引入一个正式对象：

- `room_to_debate_handoff_packet`

它至少应包含：

1. 原始议题
2. 当前房间标题
3. 主类型与副类型
4. 当前已确定的子问题
5. 已达成共识
6. 核心分歧
7. 仍未解决的问题
8. 当前已有的候选方案
9. 当前房间中已出现的重要事实
10. 已显式表达的不确定项
11. 建议带入 `/debate` 的参会 Agent
12. 建议这些 Agent 的职责草案
13. 升级原因

29.5 Handoff Packet 的作用

它的作用不是替代 `/debate`，而是让 `/debate` 不用从零开始识别问题。

换句话说：

- `/room` 负责把问题推演清楚
- `handoff packet` 负责把推演结果转成正式会议输入
- `/debate` 负责在这个基础上进行正式审议与审查

29.6 升级后的职责边界

一旦进入 `/debate`：

- `/room` 中自由群聊式的发言机制停止
- 正式切换到 `/debate` 的分工、发言、汇总、审查结构
- `/debate` 不直接继承 `/room` 的松散发言，而是只继承经过主持器整理后的输入包

29.7 防止污染的关键规则

为了不让 `/room` 污染 `/debate`，建议设定：

- `/debate` 只读取 `handoff packet`，不直接逐条消费整个群聊日志
- 群聊日志可以作为附录参考，但不应成为正式审查对象
- 正式审查对象仍然应是 `/debate` 内产生的显式发言与汇总

这样做的原因是：

- `/room` 允许探索性、试探性、不完全收敛的内容
- `/debate` 必须面对正式审查与最终决议

两者不能混为一谈。

29.8 升级时的默认选人策略

升级到 `/debate` 时，不建议机械沿用 `/room` 当前所有成员。

更合理的方式是：

- 从 `handoff packet` 中读取“建议候选人”
- 再按 `/debate` 的正式路由和对冲规则重新确定最终名单
- 允许保留一部分 `/room` 常驻成员
- 允许替换掉那些只适合探索、不适合正式裁决的成员

29.9 升级结果的用户体验

建议升级时对用户呈现为：

- “当前房间已经进入定案阶段，建议升级到 `/debate`。”
- “我将把当前讨论压缩为正式议题输入包，并重新组织圆桌审议。”

这样用户会明确感知到：

- 不是又开了一个新话题
- 而是当前讨论进入了更高严肃度阶段

---

30. 新增的整体开发建议

在完成本次补充后，下一轮开发建议的优先级进一步收敛为：

1. 把本报告中 `/room` 的发言机制、换人机制、升级协议拆成独立文档
2. 为 `/room` 新增 prompt 文档与命令说明
3. 设计 Agent Registry 的元数据标准与自动发现流程
4. 再决定是否需要真正的状态持久化实现层

这样可以确保整个系统先把“协议层和文档层”做稳，再考虑更重的工程实现。